JPEG Compression of a 24bpp Image

Student Name: Bandara LHMMD
Index number: 230078B

Introduction

A 24bpp RGB image uses 8 bits for each colour: red, green and blue. So each pixel takes 3 bytes. Before compression, an 800 x 600 image needs 1,440,000 bytes for its pixels.

JPEG makes the image file smaller by removing some small details and storing the rest using fewer bits. The image keeps the same width and height. But some data is lost, so the original pixel values cannot be fully recovered.

Processing steps

1. Convert RGB to YCbCr

Y represents brightness. Cb and Cr represent colour differences. This separates brightness from colour, so they can be compressed differently.

The conversion is approximately:

- $Y = 0.299R + 0.587G + 0.114B$
- $Cb = 128 - 0.168736R - 0.331264G + 0.5B$
- $Cr = 128 + 0.5R - 0.418688G - 0.081312B$

For RGB (100, 150, 200), the values are approximately Y = 140.75, Cb = 161.44 and Cr = 98.93. Doing just this conversion does not reduce the number of values.

2. Reduce colour samples

Our eyes can notice small brightness changes more than small colour changes. JPEG can keep all Y samples and use fewer Cb and Cr samples. In 4:2:0 sampling, a 2 x 2 area has four Y samples, one Cb sample and one Cr sample. This gives six values instead of twelve. This step is optional and loses some colour detail.

3. Divide into blocks and apply DCT

Each component is divided into 8 x 8 blocks. Extra samples are added at the edges if needed. Before the transform, 128 is subtracted from each sample. So, for example, 150 becomes 22 and therefore the range becomes -128 to +127. (-128 is black and +127 is white)

The DCT turns each block into 64 coefficients. The first is the DC coefficient. It represents the average level of the block. The other 63 are AC coefficients. They represent changes within the block.

For a constant block with Y = 140.75, the shifted value is 12.75. Its DC coefficient is 8 x 12.75 = 102. All AC coefficients are zero.

4. Quantize the coefficients

Each coefficient is divided by its matching value in an 8 x 8 quantization table. The result is rounded to an integer.

For example, 102 / 16 becomes 6. A small coefficient such as 3 / 20 becomes 0. Larger divisors usually mean smaller files and more detail lost. Fine detail is usually given larger divisors. This is the main lossy step.

5. Read in zigzag order

The 8 x 8 result is read in a zigzag pattern. This puts the slower changes first and the finer changes later. After quantization, many of the later values are zero. So we can denote those as 0x[16] instead of writing 16 zeroes.

6. Encode the values
The DC value is stored as a difference from the previous block of the same component. For DC values 6 and 8, the second difference is 2.

For AC values, runs of zeros are counted. For example, 0, 0, 5 becomes a run of two zeros followed by 5. An end-of-block symbol represents the remaining zeros. Also, Huffman coding gives shorter codes to common symbols.

7. Write the JPEG file
The encoded data is saved with the image dimensions and the required tables. To display it, the decoder reverses the coding, multiplies by the quantization values, applies inverse DCT and converts back to RGB. Obviously, details lost/removed during quantization and colour sampling can not be restored.

Native Python example [GitHub/thisismalindu - JPEG Compression]
The short program below shows the DCT and quantization calculations for one Y block. It uses only Python's math module. Inspired by Robert Ancell's PyJPEG project complete encoder written in just Python.

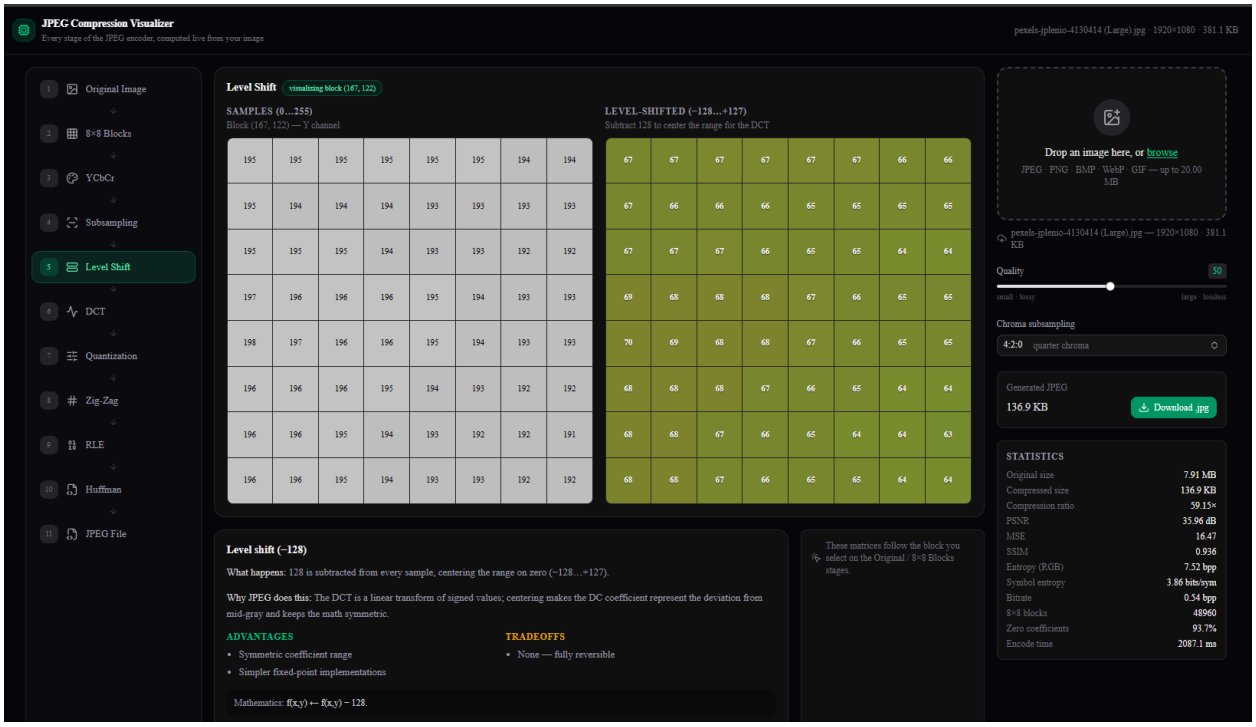
```
from math import cos, pi, sqrt

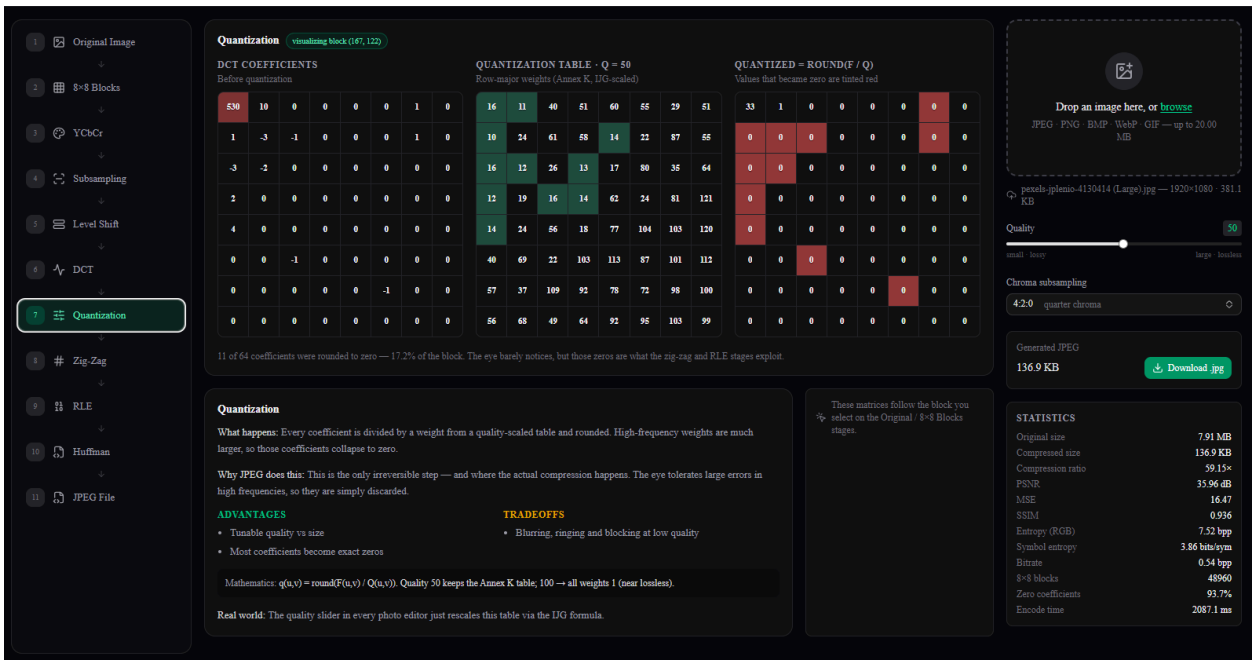
def compress_y_block(rgb):
    block = [[0.299*r + 0.587*g + 0.114*b - 128
              for r, g, b in row] for row in rgb]
    q = [
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68, 109, 103, 77],
        [24, 35, 55, 64, 81, 104, 113, 92],
        [49, 64, 78, 87, 103, 121, 120, 101],
        [72, 92, 95, 98, 112, 100, 103, 99]
    ]
    result = [[0]*8 for _ in range(8)]
    for u in range(8):
        for v in range(8):
            cu = 1/sqrt(2) if u == 0 else 1
            cv = 1/sqrt(2) if v == 0 else 1
            total = 0
            for x in range(8):
                for y in range(8):
                    total += (block[x][y]
                              * cos((2*x+1)*u*pi/16)
                              * cos((2*y+1)*v*pi/16))
            dct = cu*cv*total/4
            result[u][v] = round(dct/q[u][v])
    return result

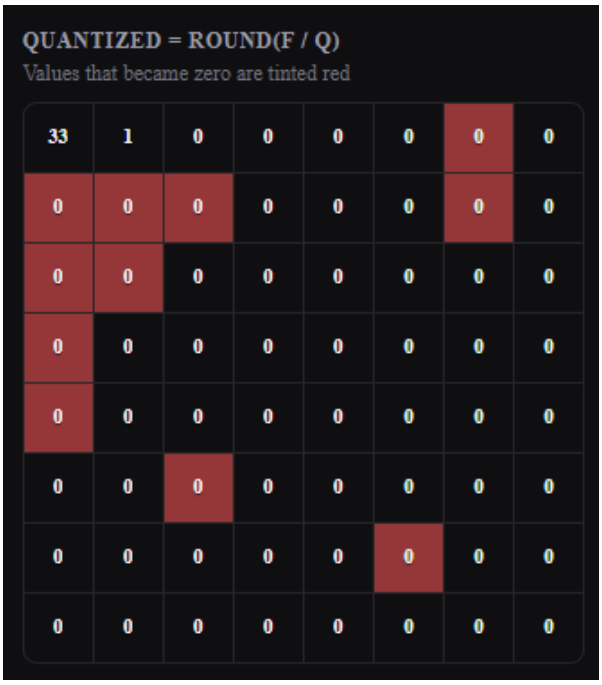
rgb = [[(100, 150, 200)]*8 for _ in range(8)]
for row in compress_y_block(rgb):
    print(row)
```

The first output row is [6, 0, 0, 0, 0, 0, 0, 0]. The remaining seven rows contain only zeros. The same DCT and quantization steps are also applied to the Cb and Cr blocks, usually with a different table.

I used a sample image on https://jpeg-compression-visualizer.vercel.app/ to verify my results. However, I had to change my code to match the website's version.








```
C:\Users\malindu\Downloads>python JR_230078B.py
[33, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
```

I modified the standard code I used to match the website's design for this example. But the basic idea remains the same. Modified code is available here: GitHub/thisismalindu - JPEG Compression

File size and quality

For an illustrative 800 x 600 image, if the JPEG file is 120,000 bytes:

- Compression ratio = 1,440,000 / 120,000 = 12:1.

The actual size depends on the image and quality setting. A grayscale JPEG is also not always ⅓ the size of a colour JPEG. Nearly gray colour images have very little colour information to store.

JPEG is useful for photographs. Strong compression can make blocks and blurred edges visible. A smaller file needs less storage, but some detail is lost. JPEG is also not good for things like storing screenshots where letters require sharp edges or compressing vector files.

References

- University of Moratuwa, Department of Computer Science and Engineering. *Unit-8-Compression_no-narration.pdf*, slides 18 and 21–24.
- Branch Education. How are Images Compressed? JPEG In Depth.
- Aishwarya K. M. JPEG Image Compression, SlideShare.
- Signal Processing Stack Exchange. How does JPEG encoding affect the final file size of 24BPP RGB and 8BPP grayscale images?.
- Stack Overflow. How can I compress a JPEG image with a compression rate of 4 bpp or less?.
- Robert Ancell. PyJPEG: a pure Python JPEG encoder and decoder.
- JPEG Compression Visualizer https://jpeg-compression-visualizer.vercel.app/